EDU-FEED

AI-Driven Intelligent Student Feedback Analysis and Insight Generation System

Baseline Models Documentation

Model 1: Sentiment Analysis | Model 2: Rating Prediction

Dataset: RateMyProfessor | Phase 2 Anonymised | Google Colab (CPU)

1. Dataset Overview

Both baseline models are built on the RateMyProfessor (RMP) dataset. This is the only dataset used at this stage. Indian college survey data will be collected later and plugged into the same pipeline.

1.1 About the Dataset

- Source: RateMyProfessors.com — a US-based public professor review website
- Available on: Kaggle (publicly available for academic research)
- Size: 20,000 rows and 51 columns before preprocessing
- After preprocessing: 19,981 rows and 45 columns

1.2 Columns Used for Baseline Models

Column Name	Used In	Purpose
comments	Model 1	Input text for sentiment classification
comment_masked	Model 1	Anonymised version of comments (Phase 2 output)
sentiment_label	Model 1	Target variable: Positive / Neutral / Negative
student_star	Model 2	Target variable: rating 1 to 5
student_difficult	Model 2	Difficulty rating feature
diff_index	Model 2	Difficulty index feature
take_again	Model 2	Would take again ratio feature
help_ratio	Model 2	Engineered: helpfulness ratio 0 to 1
tag_count	Model 2	Engineered: number of tags per professor
21 binary tag cols	Model 2	e.g. gives_good_feedback, tough_grader, caring etc.
experience_bucket	Model 2	Engineered: professor experience in years (grouped)
difficulty_category	Model 2	Engineered: Easy / Moderate / Difficult

1.3 Columns Dropped Before Models Run

- gender, race, asian, hispanic, nh_black, nh_white — dropped for privacy

- professor_name, school_name, state_name, local_name — not needed for modelling
- IsCourseOnline — replaced by course_mode engineered feature, then dropped

2. Phase 2 — Anonymisation Layer

Phase 2 runs before any model sees the data. It is a mandatory security checkpoint. Data enters Phase 2 dirty and exits clean and anonymous. Three steps are applied in sequence.

2.1 Step 1 — PII Stripping

All directly identifying columns are dropped immediately. A random UUID is assigned to each row so rows can still be counted without any link to a real person.

```
# Drop all PII columns
pii_cols = ['gender','race','asian','hispanic','nh_black','nh_white',
            'professor_name','school_name','state_name','local_name']
df.drop(columns=[c for c in pii_cols if c in df.columns], inplace=True)

# Assign anonymous UUID
from uuid import uuid4
df['survey_id'] = [str(uuid4()) for _ in range(len(df))]
```

2.2 Step 2 — K-Anonymity Check

Even without names, a student can be re-identified if they are the only person in a very small group. K-anonymity suppresses any group with fewer than 5 responses. The group is defined as department + year of review. Groups with less than 5 rows are removed from all outputs, dashboards, and model training entirely.

```
# K-Anonymity: suppress groups with fewer than 5 responses
group_col = ['department_name', 'year_since_first_review']
group_sizes = df.groupby(group_col)['survey_id'].transform('count')
df = df[group_sizes >= 5]
df.reset_index(drop=True, inplace=True)
```

2.3 Step 3 — NER Masking on Comment Text

Students sometimes write names inside their open-text comments, for example 'Prof Sharma is great' or 'my roll is 21CS045'. spaCy Named Entity Recognition (NER) reads every comment and replaces PERSON and ORG entity mentions with [NAME]. Roll numbers and emails are caught by regex patterns. The original comment is deleted after masking — only the masked version is stored and passed to all models.

- Algorithm used: Named Entity Recognition (NER) — spaCy en_core_web_sm pre-trained model
- No custom training needed — spaCy's model already detects names out of the box
- Regex additionally catches structured patterns like roll numbers and emails

```
import spacy, re
nlp = spacy.load('en_core_web_sm')

def mask_comment(text):
    if pd.isna(text) or text == '': return ''
    doc = nlp(str(text))
    for ent in doc.ents:
        if ent.label_ in ['PERSON', 'ORG']:
            text = text.replace(ent.text, '[NAME]')
    text = re.sub(r'\b\d{2}[A-Z]{2}\d{3,4}\b', '[ID]', text)
    text = re.sub(r'[\w.]+@[\w.]+\.\w+', '[EMAIL]', text)
    return text

df['comment_masked'] = df['comments'].apply(mask_comment)
df.drop(columns=['comments'], inplace=True)  # original deleted
df.to_csv('edufeed_phase2_clean.csv', index=False)
```

3. Baseline Model 1 — Sentiment Analysis

3.1 What This Model Does

This model reads a student comment and classifies it as Positive, Neutral, or Negative. It is the foundation of the EDU-FEED feedback analysis system. The sentiment label assigned to each comment is used as the target variable for the more advanced BERT model built in the next phase.

3.2 Dataset Used

- File: edufeed_phase2_clean.csv (Phase 2 output)
- Input column: comment_masked (anonymised comments)
- Target column: sentiment_label (Positive / Neutral / Negative)
- Rows used: all rows where comment_masked is not empty

3.3 How sentiment_label Was Created

The target variable sentiment_label was created during preprocessing (Step 6 — Feature Engineering) based on the student_star column:

- student_star >= 4.0 → Positive
- student_star >= 3.0 → Neutral
- student_star < 3.0 → Negative

3.4 Two Models Built

	VADER	Logistic Regression
Type	Rule-based (no training)	Machine learning
Training needed	No	Yes — 80/20 train/test split
Input	Raw masked comment text	TF-IDF vectors of comment text
How it works	Scores each word as positive or negative using a built-in dictionary	Learns patterns from training data and predicts on unseen text
Expected accuracy	55 to 65 percent	70 to 78 percent
Purpose	Quick benchmark, no setup	ML baseline to beat with BERT later

3.5 How TF-IDF Works

TF-IDF stands for Term Frequency — Inverse Document Frequency. It converts each comment into a row of numbers. Words that appear often in one comment but rarely across all comments get a high score — these are the most informative words. Common words like 'the' and 'is' get a low score and are ignored.

- max_features = 5000: only top 5000 most important words are kept
- ngram_range = (1, 2): single words and two-word phrases are both captured
- stop_words = english: common English words are removed automatically

3.6 Full Code

```
!pip install vaderSentiment spacy -q
!python -m spacy download en_core_web_sm -q

import pandas as pd
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# Load Phase 2 clean data
df = pd.read_csv('edufeed_phase2_clean.csv')
df = df[df['comment_masked'].notna() & (df['comment_masked'] != '')]
df = df[df['sentiment_label'].notna()]
df.reset_index(drop=True, inplace=True)

# ── VADER ──────────────────────────────────────────
analyzer = SentimentIntensityAnalyzer()

def vader_predict(text):
    score = analyzer.polarity_scores(str(text))['compound']
    if score >= 0.05:  return 'Positive'
    if score <= -0.05: return 'Negative'
    return 'Neutral'

df['vader_pred'] = df['comment_masked'].apply(vader_predict)
vader_acc = accuracy_score(df['sentiment_label'], df['vader_pred'])
print(f'VADER Accuracy: {vader_acc:.4f}')
print(classification_report(df['sentiment_label'], df['vader_pred']))

# ── LOGISTIC REGRESSION ────────────────────────────
X = df['comment_masked'].astype(str)
y = df['sentiment_label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2),
                        stop_words='english')
X_train_v = tfidf.fit_transform(X_train)
X_test_v  = tfidf.transform(X_test)
```

```
lr = LogisticRegression(max_iter=1000, random_state=42,
                        class_weight='balanced')
lr.fit(X_train_v, y_train)
y_pred = lr.predict(X_test_v)
lr_acc = accuracy_score(y_test, y_pred)
print(f'Logistic Regression Accuracy: {lr_acc:.4f}')
print(classification_report(y_test, y_pred))
```

3.7 Evaluation Metrics Explained

- Accuracy: percentage of total predictions that were correct
- Precision: of all comments predicted as Positive, how many were actually Positive
- Recall: of all actually Positive comments, how many did the model correctly find
- F1 Score: harmonic mean of precision and recall — the best single number to compare models
- Confusion Matrix: table showing correct vs wrong predictions for each class

3.8 Why Two Models

VADER is the rule-based benchmark — fast and needs no training. Logistic Regression is the machine learning benchmark. When the advanced BERT model is built in the next phase, its accuracy is compared against both of these to prove improvement. This comparison is what validates the need for a deep learning model.

4. Baseline Model 2 — Rating Prediction

4.1 What This Model Does

This model predicts the overall student rating (1 to 5 stars) a professor will receive based on structured feedback features. It answers the question: given what students said about teaching style, workload, and accessibility, what overall rating would they give? The model also produces a feature importance chart showing which factors matter most to students.

4.2 Dataset Used

- File: edufeed_phase2_clean.csv (same Phase 2 output)
- Target column: student_star (numeric 1.0 to 5.0)
- Input features: Likert columns + engineered features + 21 binary tag columns

4.3 Input Features

Feature	Type	Description
student_difficult	Numeric	How difficult students found the course
diff_index	Numeric	Overall difficulty index score
take_again	Numeric	Ratio of students who would take again
help_ratio	Engineered	help_useful / total help votes (0 to 1)
tag_count	Engineered	Number of tags the professor has
word_comment	Numeric	Number of words in the comment
difficulty_category	Encoded	Easy=0, Moderate=1, Difficult=2
experience_bucket	Encoded	0-2yrs=0, 3-5yrs=1, 6-10yrs=2, 10+yrs=3
21 binary tag cols	Binary 0/1	gives_good_feedback, tough_grader, caring etc.

4.4 Two Models Built

- Linear Regression — simple baseline, assumes a straight-line relationship between features and rating

- Random Forest — ensemble of decision trees, captures non-linear patterns, produces feature importance

4.5 Evaluation Metrics

- RMSE (Root Mean Square Error): average prediction error in star units. Lower is better. Target < 0.8
- MAE (Mean Absolute Error): average absolute difference between predicted and actual rating
- R-squared: how much of the rating variation the model explains. Higher is better. Target > 0.5

4.6 Full Code

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

# Load Phase 2 clean data
df = pd.read_csv('edufeed_phase2_clean.csv')
df = df[df['student_star'].notna()].reset_index(drop=True)

# Encode categorical features
le = LabelEncoder()
df['difficulty_cat_enc']  =
le.fit_transform(df['difficulty_category'].astype(str))
df['experience_enc']      =
le.fit_transform(df['experience_bucket'].astype(str))

# Define features
tag_cols =
['gives_good_feedback','caring','respected','participation_matters',

'clear_grading_criteria','skip_class','amazing_lectures','inspirational',

'tough_grader','hilarious','get_ready_to_read','lots_of_homework',
            'accessible_outside_class','lecture_heavy','extra_credit',
            'graded_by_few_things','group_projects','test_heavy',
            'so_many_papers','beware_of_pop_quizzes']

feature_cols = ['student_difficult','diff_index','take_again','help_ratio',
                'tag_count','word_comment','difficulty_cat_enc',
                'experience_enc'] + tag_cols

X = df[feature_cols].fillna(0)
```

```
y = df['student_star']

# Train test split 80/20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# ── LINEAR REGRESSION ──────────────────────────────
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

lr_rmse = np.sqrt(mean_squared_error(y_test, y_pred_lr))
lr_mae  = mean_absolute_error(y_test, y_pred_lr)
lr_r2   = r2_score(y_test, y_pred_lr)
print(f'Linear Regression  RMSE:{lr_rmse:.3f}  MAE:{lr_mae:.3f}
R2:{lr_r2:.3f}')

# ── RANDOM FOREST ──────────────────────────────────
rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

rf_rmse = np.sqrt(mean_squared_error(y_test, y_pred_rf))
rf_mae  = mean_absolute_error(y_test, y_pred_rf)
rf_r2   = r2_score(y_test, y_pred_rf)
print(f'Random Forest      RMSE:{rf_rmse:.3f}  MAE:{rf_mae:.3f}
R2:{rf_r2:.3f}')

# ── FEATURE IMPORTANCE PLOT ────────────────────────
importances = pd.Series(rf.feature_importances_, index=feature_cols)
top10 = importances.nlargest(10).sort_values()

plt.figure(figsize=(8, 5))
top10.plot(kind='barh', color='steelblue', edgecolor='black')
plt.title('Top 10 Features — Random Forest', fontweight='bold')
plt.xlabel('Feature Importance')
plt.tight_layout()
plt.show()

# ── COMPARISON TABLE ───────────────────────────────
print(f'{'Model':<22} {'RMSE':>8} {'MAE':>8} {'R2':>8}')
print('-' * 48)
print(f'{'Linear Regression':<22} {lr_rmse:>8.3f} {lr_mae:>8.3f}
{lr_r2:>8.3f}')
print(f'{'Random Forest':<22} {rf_rmse:>8.3f} {rf_mae:>8.3f} {rf_r2:>8.3f}')
```

4.7 What the Feature Importance Chart Tells You

The feature importance chart from Random Forest shows which input features had the most influence on predicting the star rating. This is one of the most important outputs of this model because it reveals what

students actually care about most — not just what we assume they care about. For the Indian college context, the equivalent chart will show which aspects of faculty performance drive overall satisfaction.

4.8 Why Random Forest Over Linear Regression

- Linear Regression assumes all features affect the rating in a straight-line way — this is rarely true in real feedback data
- Random Forest builds 100 decision trees and averages their predictions — much better at capturing complex patterns
- Random Forest also gives feature importance which Linear Regression does not
- Linear Regression is kept as the baseline to prove that Random Forest actually improves results

5. How the Two Models Connect to the Full Pipeline

These two baseline models are not standalone experiments — they are the foundation that the rest of the EDU-FEED pipeline builds on.

- **establishes the text classification pipeline. VADER sets the floor. Logistic Regression sets the ML baseline. BERT in Phase 4 will be compared against both. The sentiment_label column created here becomes the target for BERT fine-tuning.**Model 1 (Sentiment Analysis)

- **produces the feature importance chart that tells us which factors drive student satisfaction. This directly informs the aspect weights used in the faculty radar chart in the dashboard. The Random Forest model can also predict expected ratings for new faculty with fewer reviews.**Model 2 (Rating Prediction)

When Indian survey data is collected, both models are retrained on that data. The RMP-trained models serve as pre-trained starting points — the pipeline does not change, only the input data changes.

Model	Input	Output	Next Phase Use
Model 1 — Sentiment	comment_masked text	Positive / Neutral / Negative label	BERT fine-tuning target
Model 2 — Rating	Likert + tag features	Predicted star rating + feature importance	Faculty radar chart weights